ANALYSIS AND DESIGN OF ALGORITHMS

Mid-Semester Mock Examination • Set 1 • Based on your college sample paper and stated syllabus

Suggested maximum marks: 100 **Suggested duration:** 2.5 hours

Note: This is a practice paper, not a prediction or guarantee of the exact exam. Marks and duration are suggested; the supplied sample does not establish the official total or duration.

Instructions

• Attempt all questions. Draw neat trees and graphs where required.

• Show intermediate steps for dry runs and Master's Theorem.

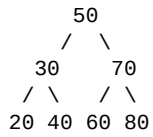
• For BFS/DFS, follow the stated neighbour order. If none is specified, state your chosen order.

• For time complexity, show the reasoning behind the final bound.

QUESTION 1 — Binary Search Tree (12 marks)

A. Deletion and insertion (6 marks)

Consider this BST:



a) Delete node 20 and draw the resulting BST. (2)
b) Delete node 30 from the original BST and draw the resulting BST. (2)
c) Insert 65 into the original BST and draw the resulting BST. (2)

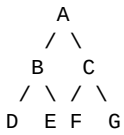
B. BST properties (6 marks)

A BST has root 100; left child 50 with children 30 and 70; right child 150 with children 120 and 180; node 70 has children 60 and 80; node 120 has right child 140.

a) If X is the left child of 70, find the minimum and maximum possible integer values of X. (2)
b) Explain all three BST deletion cases with an example each. (4)

QUESTION 2 — Tree Traversals and Fundamentals (12 marks)

A. Traversals (6 marks) For the tree below, write In-order, Pre-order and Post-order traversals.



B. MCQs (6 marks, 1 mark each)

1. Which traversal of a BST produces sorted ascending order? (A) Pre-order (B) In-order (C) Post-order (D) Level-order

2. In a zero-indexed heap, the parent of element at index i is: (A) 2i+1 (B) 2i+2 (C) floor((i–1)/2) (D) i–1

3. The primary purpose of a recursive base case is to: (A) Increase recursion depth (B) Stop recursion (C) Sort an array (D) Create an object

4. Worst-case Quick Sort complexity: (A) O(log n) (B) O(n) (C) O(n log n) (D) $O(n^2)$

5. Which data structure is normally used by BFS? (A) Stack (B) Queue (C) Heap (D) BST

6. Adjacency-list space for V vertices and E edges: (A) $O(V^2)$ (B) O(V+E) (C) $O(E^2)$ (D) O(1)

QUESTION 3 — Time Complexity (10 marks)

A. Find and explain the time complexity of each snippet. (6 marks)

i) (2 marks)

```
for i in range(n):
    print(i)
```

ii) (2 marks)

```
i = 1
while i < n:
    i *= 2
```

iii) (2 marks)

```
for i in range(n):
    for j in range(n):
        print(i, j)
```

B. (4 marks) Arrange these from smallest to largest growth rate and briefly explain: $O(n^2)$, $O(1)$, $O(n \log n)$, $O(\log n)$, $O(n)$.

QUESTION 4 — Quick Sort (10 marks)

A. Theory (4 marks) Explain divide-and-conquer, pivot, partitioning, recursive calls, and average/worst-case time complexity.

B. Dry run (6 marks) Apply Quick Sort to [7, 3, 8, 2, 5] using the last element as pivot and the Lomuto partition scheme. Show the first partition, subsequent subarrays/pivots, and final sorted array.

QUESTION 5 — Heap (12 marks)

A. Heap operations (6 marks) Starting with an empty max-heap, insert [40, 20, 35, 10, 15, 30, 50]. Show the array after every insertion, including swaps.

B. Deletion (3 marks) Delete the maximum from the final heap. Show replacement, sift-down, and final array.

C. Theory (3 marks) State complexities for heap insertion, delete-max, and build-heap. Explain why build-heap is O(n), not O(n log n).

QUESTION 6 — Master's Theorem (18 marks)

A. State the theorem (3 marks) For T(n) = aT(n/b) + f(n), define a, b, f(n), and the critical term n^(log_b a).

B. Solve each recurrence (9 marks, 3 each) Show a, b, f(n), critical term, case, and final complexity.

1. $T(n) = 2T(n/2) + n$

2. $T(n) = 2T(n/2) + n^2$

3. $T(n) = 4T(n/2) + n$

C. (3 marks) Solve $T(n) = 3T(n/2) + n$ using Master's Theorem and state the case.

D. (3 marks) Find the complexity and justify:

```
for i in range(n):
    j = 1
    while j < n:
        j *= 2
```

QUESTION 7 — Backtracking and Dry Run (12 marks)

A. Dry run (7 marks) Trace the complete function calls, path state, call-stack progression, and exact console output.

```
def explore(x, target, path):
    path.append(x)
    print(f"Visited: {x}, Current Path: {path}")
```

```
    if x == target:
        print(f"Target {target} reached!")
        return True
    for next_val in [x + 1, x + 2]:
        if next_val <= target:
            if explore(next_val, target, path):
                return True
    popped = path.pop()
    print(f"Backtracking from: {popped}")
    return False

explore(1, 3, [])
```

B. Theory (5 marks) 1) Define backtracking and its relationship with recursion. (2) 2) Explain the base case and recursive choice. (2) 3) Why is path.pop() not reached when the target is found? (1)

QUESTION 8 — Graphs: BFS and DFS (14 marks)

Consider this undirected graph. Visit neighbours in alphabetical order.

```
Edges: A–B, A–C, B–C, B–D, C–E, D–E
```

A. (4 marks) Write the adjacency list and adjacency matrix in the order A, B, C, D, E.

B. (4 marks) Perform BFS from A. Show the queue after each iteration and traversal order.

C. (4 marks) Perform DFS from A using alphabetical neighbour order. Show traversal order and recursion/stack progression.

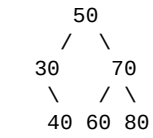
D. (2 marks) Differentiate BFS and DFS by data structure, traversal strategy, and one application of each.

ANSWER KEY & EXPLANATIONS

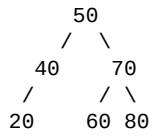
Use this section after attempting the paper. Alternative valid BST deletion choices and valid traversal orders may exist where the question permits them.

Q1 — BST

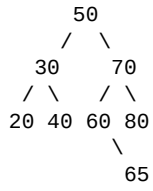
A(a) Delete 20: 20 is a leaf, so remove it.



A(b) Delete 30: 30 has two children. Using inorder successor 40, replace 30 with 40 and remove the original 40.



A(c) Insert 65: 65 > 50, 65 < 70, 65 > 60, so it becomes the right child of 60.



B(a): As left child of 70, X must satisfy 50 < X < 70. Distinct integer bounds: minimum 51, maximum 69.

B(b): Leaf: remove directly. One child: link parent to child. Two children: replace with inorder successor or predecessor, then remove that replacement from its old position. Either successor or predecessor is valid if BST ordering is preserved.

Q2 — Traversals and MCQs

In-order: D B E A F C G
Pre-order: A B D E C F G
Post-order: D E B F G C A

MCQ	Answer	Reason
1	B — In-order	BST keys are visited in ascending order.
2	C — floor((i–1)/2)	Parent index in a zero-indexed binary heap.
3	B — Stop recursion	Base case prevents unbounded recursive calls.
4	D — $O(n^2)$	Quick Sort's worst case can be quadratic.
5	B — Queue	BFS is FIFO.
6	B — O(V+E)	Adjacency lists store vertices and edge references.

Q3 — Time Complexity

A(i): O(n), because the loop runs n times.

A(ii): O(log n), because i doubles each iteration: 1, 2, 4, 8, …, and reaches n after about log■n iterations.

A(iii): $O(n^2)$, because n outer iterations each run n inner iterations.

B: O(1) < O(log n) < O(n) < O(n log n) < O(n^2).

Q4 — Quick Sort

Theory: Quick Sort is divide-and-conquer: choose a pivot, partition elements around it, then recursively sort the subarrays. After partitioning, the pivot is in its final sorted position. Average/best: O(n log n); worst: O(n^2).

Dry run, Lomuto partition with last element as pivot:

```
Start: [7, 3, 8, 2, 5]
Pivot 5; partition gives [3, 2, 5, 7, 8]
Sort left [3, 2], pivot 2 -> [2, 3]
Sort right [7, 8], pivot 8 -> [7, 8]
Final: [2, 3, 5, 7, 8]
```

For the first partition: 3 is moved left; 2 is then moved left; pivot 5 swaps into index 2.

Q5 — Heap

A. Max-heap after each insertion:

Inserted	Heap array
40	[40]
20	[40, 20]
35	[40, 20, 35]
10	[40, 20, 35, 10]
15	[40, 20, 35, 10, 15]
30	[40, 20, 35, 10, 15, 30]
50	[50, 20, 40, 10, 15, 30, 35]

For insertion of 50: append at the end, swap with 35, then swap with 40.

B. Delete maximum: Remove 50; move last value 35 to the root. Compare 35 with children 20 and 40, then swap with 40. Final array: [40, 20, 35, 10, 15, 30].

C. Insert O(log n); delete-max O(log n); build-heap O(n). Build-heap is linear because most nodes are near leaves and need little sift-down work; not every node incurs log n work.

Q6 — Master's Theorem

For T(n) = aT(n/b) + f(n), a is the number of subproblems, b is the shrink factor, and f(n) is non-recursive work. Compare f(n) with n^(log_b a).

Case	Comparison	Result (standard conditions)
1	f(n) polynomially smaller	Theta(n^(log_b a))
2	f(n) matches critical term (up to log factors)	Theta(n^(log_b a) log n) for f(n)=Theta(n^(log_b a))
3	f(n) polynomially larger and regularity condition holds	Theta(f(n))

1. T(n)=2T(n/2)+n: a=2, b=2, f(n)=n; critical term n^(log■2)=n. Case 2: Θ(n log n).

2. T(n)=2T(n/2)+n^2: critical term n; f(n)=n^2 is polynomially larger. Case 3 (regularity holds): Θ(n^2).

3. T(n)=4T(n/2)+n: critical term n^(log■4)=n^2; f(n)=n is polynomially smaller. Case 1: Θ(n^2).

C. T(n)=3T(n/2)+n: critical term n^(log■3) ≈ n^1.585; f(n)=n is polynomially smaller. Case 1: Θ(n^(log■3)).

D. Nested loops: outer loop is n; inner loop doubles j, so O(log n). Total O(n log n).

Q7 — Backtracking

Calls: explore(1,3,[]) → explore(2,3,[1]) → explore(3,3,[1,2]).

At the target call, path becomes [1, 2, 3]. Since x == target, it prints the target message and returns True. That True propagates back through callers.

Exact output:

```
Visited: 1, Current Path: [1]
Visited: 2, Current Path: [1, 2]
Visited: 3, Current Path: [1, 2, 3]
Target 3 reached!
```

There is no backtracking output because each caller immediately returns True when the recursive call succeeds, so path.pop() is not executed.

Theory: Backtracking explores choices recursively and abandons a path when it cannot or need not continue. The base case is x == target; choices are x+1 and x+2 when they do not exceed target.

Q8 — Graphs

A. Adjacency list:

```
A: B, C
B: A, C, D
C: A, B, E
D: B, E
E: C, D
```

Adjacency matrix (A, B, C, D, E):

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	1	1	0
C	1	1	0	0	1
D	0	1	0	0	1
E	0	0	1	1	0

B. BFS from A (alphabetical neighbours):

Processed	Queue after processing
A	[B, C]
B	[C, D]
C	[D, E]
D	[E]
E	[]

BFS order: A, B, C, D, E.

C. DFS from A: A → B → C → E → D, then backtrack as no unvisited neighbours remain.

D. BFS uses a queue and explores level by level; it finds shortest paths in unweighted graphs. DFS uses a stack or recursion and explores deeply before backtracking; it is useful for cycle detection and connected components.

Final revision priorities

• Master's Theorem: identify a, b, f(n), critical term, and case.

- Quick Sort: perform partitioning without skipping swaps.
- Heap: remember sift-up, sift-down, and O(n) build-heap.
- BST: practise all three deletion cases and traversals.
- Backtracking: trace path, call stack, return values, and exact output.
- Graphs: practise adjacency representations, BFS queue, and DFS recursion.
- Complexity: recognise single, nested, and doubling/halving loops.